



AI Mode All Videos Images Forums Short videos News Web Books Finance

in Golang when in a source file you add a go generate comment, is it tied to what immediately follows in the file? ▾

5 sites ▮

No, `go generate` comments are **not** tied to what immediately follows them in the source file. [↗](#)

The `go generate` tool simply scans your `.go` files line-by-line looking for the specific magic comment string. When it finds one, it runs the command specified on that line, completely ignoring any variables, functions, or structs written below it. [↗](#)

Key Rules of `go generate`

- **File-Level Scope:** The command runs within the directory containing the file.
- **No Direct Association:** You can place the comment at the very top of an empty file, inside a function, or at the bottom of the file.
- **Exact Syntax Required:** The comment must start exactly with `//go:generate` (no space after the slashes).
- **Independent Execution:** It does not look at surrounding Go AST (Abstract Syntax Tree) nodes unless the tool you are calling specifically accepts arguments telling it to scan the current file or package (like `stringer`). [↗](#)

Common Best Practice

Even though Go doesn't enforce an association, human developers do. It is highly recommended to place the `go generate` comment **directly above** the type or function it relates to so that other programmers can easily see why the generator is being called. [↗](#)

```
go
package main

// Good practice: The comment is placed right above the type it modifies,
// even though the go tool reads it independently.
//go:generate stringer -type=Pill
type Pill int
```

Use code with caution.



Would you like help **writing a custom generator tool**, or are you trying to debug a specific `go generate` command that isn't running properly?

AI responses may include mistakes. [Learn more](#)



what if you have multiple go generate in a single file? will the output then get appended to a single generated go file? ▾

3 sites ▮

Ask anything



The `go generate` tool does not capture, combine, or write any output itself. It simply runs the commands you specify as separate shell processes. Where the output goes depends entirely on the specific tool you are calling. [↗](#)

How Different Tools Handle Output

- **Tools with internal file creation:** Many tools (like `stringer` or `protobuf`) automatically create their own separate files based on their arguments. They do not append.
- **Tools using standard output (`>`):** If you use shell redirection to write to the same file, the behavior depends on the operator you use.

Examples of What Happens

Scenario 1: Using tools that manage their own files

```
go

package main

//go:generate stringer -type=Poll
//go:generate stringer -type=Status
```

Use code with caution.



- **Result:** This will create **two separate files**: `poll_string.go` and `status_string.go`.

Scenario 2: Overwriting with `>`

```
go

package main

//go:generate echo "package main" > generated.go
//go:generate echo "var X = 1" > generated.go
```

Use code with caution.



- **Result:** The second command will **overwrite** the first. `generated.go` will only contain `var X = 1`, resulting in a broken, packageless Go file.

Scenario 3: Appending with `>>`

```
go

package main

//go:generate echo "package main" > generated.go
//go:generate echo "var X = 1" >> generated.go
```

Use code with caution.



- **Result:** This **will append** the output. `generated.go` will successfully contain both lines because the shell append operator (`>>`) was explicitly used.

Execution Order

When a file contains multiple `go generate` directives, they are always executed sequentially from **top to bottom** within that file. [↗](#)



AI responses may include mistakes. [Learn more](#)



so the generator should possibly be file aware and do the appending

 5 sites



Yes, exactly. If you want a clean, unified file output from multiple types, the generation tool itself must be smart enough to handle that. [🔗](#)

Instead of writing multiple `go generate` lines that conflict, the standard Go convention is to call the generator **once per package** or **once per file**, passing multiple targets as arguments to that tool.

The Right Way: Pass Multiple Arguments to One Tool

Most well-written Go generators are designed to accept multiple types at once. They will read all the types you give them, bundle the code together internally, and write a single, clean file.

```
go
package main

// CORRECT: Call the tool once and pass both types.
// The tool will merge them into a single file.
//go:generate stringer -type=Pill,Status
```

Use code with caution. 

- **Result:** The `stringer` tool creates exactly **one file** named `pill_string.go` that contains the string methods for *both* `Pill` and `Status`. [🔗](#)

How a Generator is "File Aware"

If you are writing your own custom generation tool, you should design it to be file or package aware using Go's built-in parsing libraries. Here is how professional Go tools handle this internally: [🔗](#)

1. **Use `go/ast` and `go/parser`:** Your generator tool should parse the entire target Go file or package into an Abstract Syntax Tree (AST).
2. **Scan for targets:** Look through the AST for all matching structs, interfaces, or types that need code generation.
3. **Buffer the output:** Write all generated code into a single internal buffer (like `bytes.Buffer`).
4. **Write once:** Flush that buffer into a single output file at the very end of the execution.

Would you like to see a minimal example of how to **write a custom generator using `go/parser`**, or are you currently trying to integrate a specific **third-party tool**?

AI responses may include mistakes. [Learn more](#)



yes



Here is a minimal, complete example of a **file-aware custom generator**.

This generator reads a target Go file, parses its Abstract Syntax Tree (AST), finds all struct definitions, and generates a single file containing a `Describe()` method for every struct it finds.

1. The Generator Tool (`main.go`)

Create a separate directory named `cmd/descgen` and place this code inside `main.go`. This tool parses a file, buffers the generated code, and writes it out once.

```
go

package main

import (
    "bytes"
    "fmt"
    "go/ast"
    "go/format"
    "go/parser"
    "go/token"
    "log"
    "os"
)

func main() {
    // The go generate tool automatically sets $GOFILE to the file containing the comment
    inputFile := os.Getenv("GOFILE")
    if inputFile == "" {
        log.Fatalf("GOFILE environment variable not set. Run this via 'go generate'.")
    }
    outputFile := "descriptions_gen.go"

    // 1. Parse the target Go file into an AST
    fset := token.NewFileSet()
    node, err := parser.ParseFile(fset, inputFile, nil, parser.ParseComments)
    if err != nil {
        log.Fatalf("failed to parse file: %v", err)
    }

    // 2. Setup a buffer to collect all generated code
    var buf bytes.Buffer
    buf.WriteString(fmt.Sprintf("// Code generated by descgen. DO NOT EDIT.\npackage %s\n\n", node.Name.Name))

    // 3. Inspect the AST to find all struct types sequentially
    structCount := 0
    ast.Inspect(node, func(n ast.Node) bool {
        // Look for type declarations
        typeSpec, ok := n.(*ast.TypeSpec)
        if !ok {
            return true
        }

        // Verify the declared type is actually a Struct
        if _, ok := typeSpec.Type.(*ast.StructType); ok {
            structName := typeSpec.Name.Name

            // Append the generated code for this struct to our single buffer
            buf.WriteString(fmt.Sprintf("func (%s) Describe() string {\n", structName))
            buf.WriteString(fmt.Sprintf("\treturn \"This is a %s struct\"\n", structName))
            buf.WriteString("}\n\n")
            structCount++
        }
    })
}
```



```

if structCount == 0 {
    fmt.Println("No structs found to generate code for.")
    return
}

// 4. Format the generated Go code nicely
formattedCode, err := format.Source(buf.Bytes())
if err != nil {
    log.Fatalf("failed to format generated code: %v", err)
}

// 5. Write everything to a single output file at the very end
err = os.WriteFile(outputFile, formattedCode, 0644)
if err != nil {
    log.Fatalf("failed to write file: %v", err)
}

fmt.Printf("Successfully generated %s containing %d structs\n", outputFile, structCount)
}

```

Use code with caution.



2. The Target File (`data.go`)

This is your application code. You install your generator, place one `go:generate` comment at the top, and let it scan the multiple structs below it.

```

go

package main

// First, compile and install your generator so the 'go' tool can find it
//go:generate go install ./cmd/descgen
// Tell go generate to run your tool
//go:generate descgen

type User struct {
    Name string
    Age  int
}

type Product struct {
    ID    string
    Price float64
}

type Order struct {
    ID int
}

```

Use code with caution.



3. How to Run It

Run the following command in your terminal:

```
bash
```



Use code with caution.



4. The Single Appended Output (`descriptions_gen.go`)

Because your generator collected all data from the AST before writing, it outputs exactly one perfectly appended file:

```
go
// Code generated by descgen. DO NOT EDIT.
package main

func (User) Describe() string {
    return "This is a User struct"
}

func (Product) Describe() string {
    return "This is a Product struct"
}

func (Order) Describe() string {
    return "This is a Order struct"
}
```

Use code with caution.



Would you like to expand this example to parse **struct field names**, or do you want to learn how to scan an **entire package directory** instead of just a single file?

AI responses may include mistakes. [Learn more](#)



would you commit the generated files to source control?
should build script generate them wach time

29 sites



In the Go ecosystem, the overwhelming best practice is to **commit generated files to source control**. Stack Overflow +1

While traditional software engineering rules usually say "never commit generated artifacts", Go handles code generation differently. The official Go team design proposal for `go generate` specifically outlines that authors should commit the generated files to the repository. Software Engineering... +2

Here is why Go projects handle this differently, along with a few rare exceptions.

Why You Should Commit Generated Files

- **Eliminates Third-Party Tool Chains:** If a user runs `go get` or clones your project to build it, they only need the Go compiler. They do not need to install your custom generator, `stringer`, `mockgen`, or `protobuf` compilers just to build the app.
- **Guarantees Reproducible Builds:** Different versions of a generation tool might output slightly different code. Committing the file ensures that everyone on your team—and your CI/CD server—compiles the exact same Go code every single time. GitHub +4

Speed Up CI/CD and Local Builds `go generate` is a compiler that doesn't even automatically delete `go generate` files...



- **Native Go Tooling Compatibility:** Go's native compilation and test commands (`go build` and `go test`) completely ignore `//go:generate` comments. They expect all code files to already exist on disk when compilation starts.  [Reddit · r/golang +2](#)

Best Practices When Committing Generated Files

If you commit your files, you should implement these safety measures:

1. **Add a "DO NOT EDIT" Header:** Always ensure the very first line of your generated file reads exactly like this:

```
go

// Code generated by YOUR_TOOL_NAME. DO NOT EDIT.
```

Use code with caution.



Editors (like VS Code or GoLand) and GitHub read this exact comment block to hide the file from code searches and warn developers if they accidentally try to change it.  [OneUptime +1](#)

2. **Add a CI Lint Check:** To make sure developers don't update a struct but forget to run `go generate` before pushing, add this step to your GitHub Actions or CI pipeline:

```
bash

go generate ./...
git diff --exit-code
```

Use code with caution.



If a developer forgot to generate files locally, the CI tool will regenerate them, see a Git diff, and fail the build.  [Go Cookbook +3](#)

When Should You NOT Commit Them? (The Exceptions)

You should only leave generated files out of source control (adding them to `.gitignore`) if:

- **The files are environment-dependent:** The code generates secrets or machine-specific file paths. 
- **The output is unmanageably massive:** The generated code spans millions of lines (e.g., massive database definitions) and bloats Git history unnecessarily. 
- **You use an advanced build system:** Your team uses a hermetic build orchestrator like **Bazel**, which inherently manages code generation dependencies perfectly across builds. 

How do you plan to handle your **CI/CD pipeline**, or would you like a **GitHub Actions example script** that automatically catches un-generated code changes?  [DEV Community +1](#)

AI responses may include mistakes. [Learn more](#)



Do you commit all the generated code in your...

26 May 2018 — As mentioned above, the go command is not a general...

 [Reddit · r/golang](#)

Should a developer commit Go generated code?

2 Jun 2019 — In general, you should commit Go generated code to versi...

 [Stack Overflow](#)

Do I check generated co...

27 Mar 2013 — In general, generated...

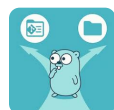
 [Software Engineering Stack...](#)



Golang: Do you commit your...

17 Jan 2023 — As mentioned above, t...

 [GitHub Pages documentation](#)



Should I con generated p

1 Sept 2013 — 2, suggestion is no

 [Software En](#)



